МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ
МЕГАФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И
УПРАВЛЕНИЯ

Отчет по лабораторной работе на тему “Разработка встроенного приложения: калькулятор на микроконтроллере с выводом на семисегментный индикатор”

Преподаватель:

Гончаров Алексей Андреевич

Работу выполнили:

Студент 3 курса

Группы P3310

Давидюк Антон Юрьевич

г. Санкт-Петербург – 2026

1. Цель и задачи работы

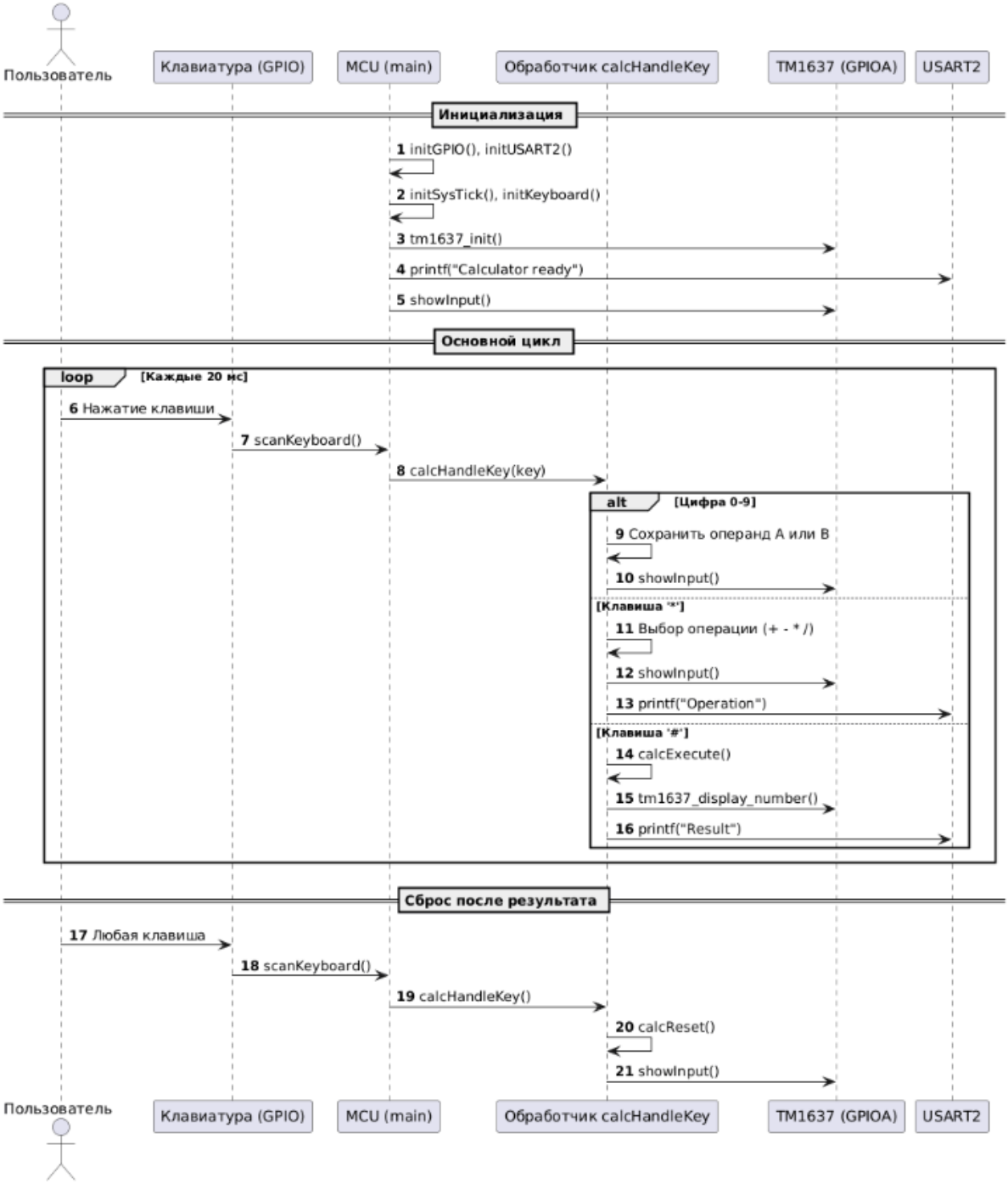
Цель работы:
Разработка и отладка программного обеспечения для микроконтроллера архитектуры ARM Cortex-M0+, реализующего функционал простого калькулятора с вводом данных через матричную клавиатуру и выводом результатов на 4-разрядный семисегментный индикатор TM1637 и в консоль UART.

Задачи работы:

1. Настроить периферийные модули микроконтроллера: GPIO (для клавиатуры и индикатора), USART2 (для отладочного вывода) и системный таймер SysTick (для временных интервалов).
2. Реализовать надежный опрос матричной клавиатуры с программным антидребезгом контактов и генерацией одиночного события нажатия.
3. Разработать конечный автомат (State Machine) для управления состояниями калькулятора (CALC_INPUT, CALC_RESULT) и обработки последовательности ввода: операнд A → операция → операнд B → вычисление.
4. Реализовать арифметические операции (+, -, *, /), включая алгоритм деления с получением дробной части (до двух знаков) исключительно средствами целочисленной арифметики, а также обработку ошибки деления на ноль.
5. Обеспечить визуализацию процесса ввода (с использованием условных обозначений операций на 7-сегментном индикаторе) и финального результата.

2. Схема взаимодействия процессора и периферийных устройств

Ниже представлена диаграмма последовательностей (Sequence Diagram), описывающая основной цикл работы программы.



3. Описание процессов, изображенных на схеме

Инициализация: При старте микроконтроллер настраивает тактирование портов GPIOA и GPIOB. Пины PA6 и PA7 настраиваются как выходы с открытым стоком для программной реализации протокола TM1637. Настраивается USART2 на скорость 115200 бод для отладки. SysTick конфигурируется на генерацию прерываний каждые 1 мс для системного времени.

Опрос клавиатуры: Функция scanKeyboard() вызывается в бесконечном цикле while(1). Она опрашивает матрицу клавиш не чаще, чем раз в 20 мс (KEY_SCAN_PERIOD_MS). Для устранения дребезга контактов требуется совпадение считанного значения в двух последовательных опросах. Функция возвращает символ нажатия только один раз.

Обработка ввода (calcHandleKey):

- При вводе цифры она записывается в operandA (если он равен -1) или в operandB (перезаписывая предыдущее значение).
- При нажатии * индекс операции opIndex циклически меняется от 0 до 3 (+, -, *, /).
- После каждого изменения состояния вызывается showInput(), которая формирует массив из 4-х байт сегментов. Для не введенных значений используется код 0x08 (сегмент 'd', нижняя черта). Для операций используются специально подобранные коды сегментов (например, 0x46 для '+').

Вычисление (calcExecute): Вызывается по нажатию #, только если введены оба операнда и выбрана операция.

- Сложение, вычитание и умножение выполняются стандартно, результат передается в tm1637_display_number(), которая выравнивает число вправо и корректно обрабатывает отрицательный знак.
- Деление реализовано без плавающей точки: формула (a * 200 + b) / (2 * b) позволяет получить результат, умноженный на 100, с математическим округлением. Затем число разбивается на целую и дробную части, и десятичная точка (бит 0x80) устанавливается в нужном разряде индикатора.
- При b == 0 вызывается showError(), выводящая на индикатор код символа "Err".

Сброс: Если калькулятор находится в состоянии CALC_RESULT, любое новое нажатие (цифра или *) автоматически вызывает calcReset(), обнуляя переменные и возвращая систему в режим ввода.

4. Руководство пользователя

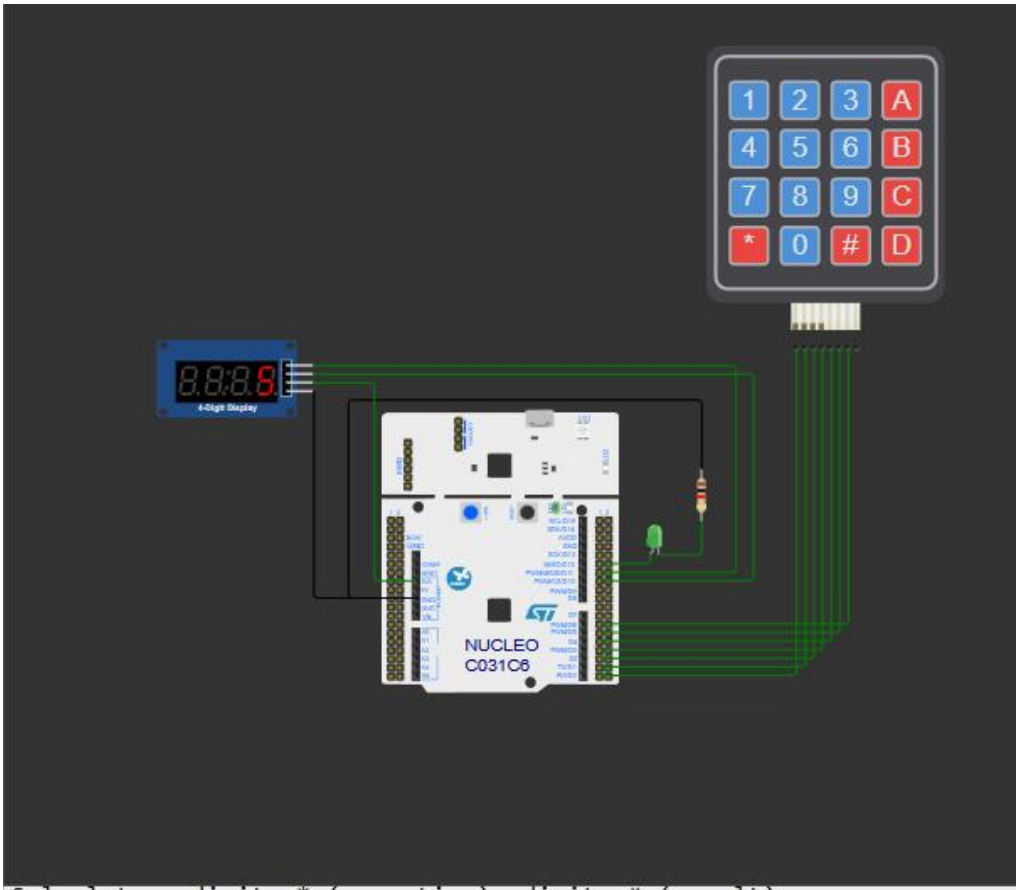
4.1. Начало работы

После прошивки микроконтроллера и подключения к терминалу UART (115200, 8N1) на индикаторе отобразится _ _ _ _, а в консоль будет выведено приветственное сообщение.

4.2. Выполнение арифметических операций

Сценарий 1: Сложение

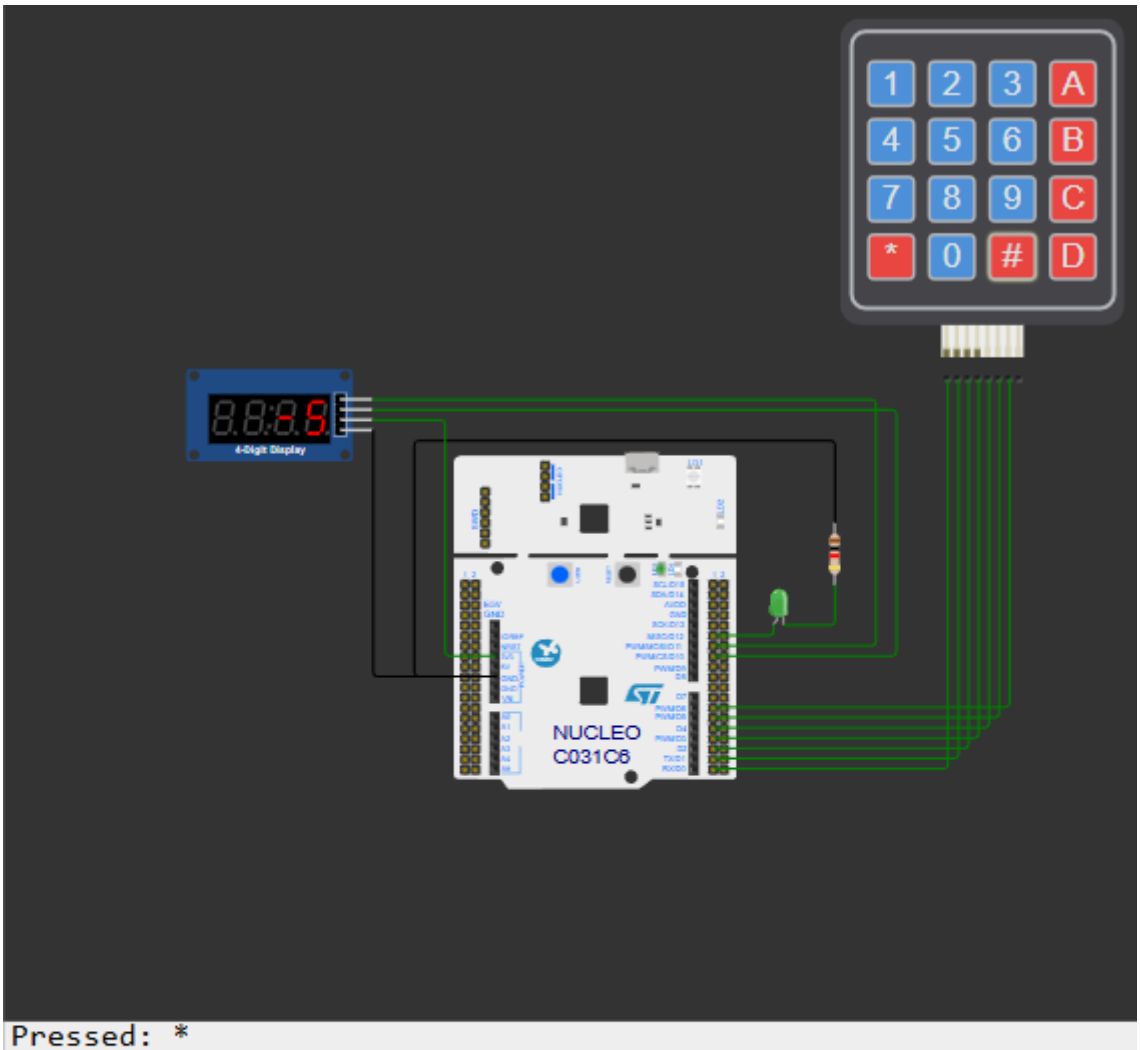
1. Нажмите 2. Индикатор: 2 _ _ _
2. Нажмите *. Индикатор: 2 + _ _ (в консоли: Operation: +)
3. Нажмите 3. Индикатор: 2 + 3 _
4. Нажмите #. Индикатор: 5 (выровнено вправо).
   Консоль: 2 + 3 = 5




```
Calculator: digit, * (operation), digit, # (result)
Pressed: 2
Pressed: *
Operation: +
Pressed: 3
Pressed: #
2 + 3 = 5
```

Сценарий 2: Умножение и отрицательный результат

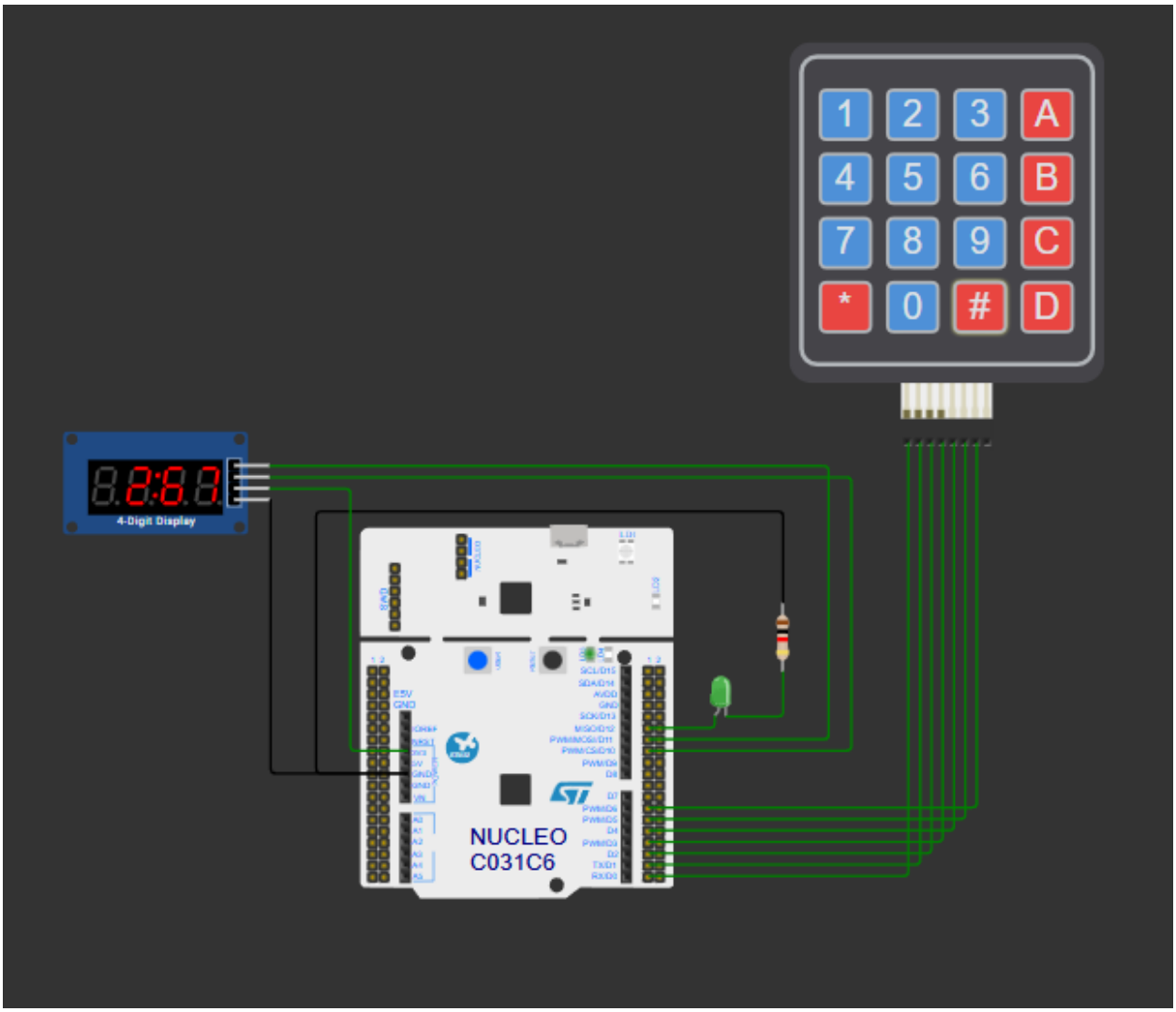
1. Нажмите 3, затем * (два раза, чтобы выбрать -), затем 8, затем #.
   Индикатор: -5 (знак минуса отображается слева).
   Консоль: Operation: - \n 3 - 8 = -5




```
Pressed: *
Operation: +
Pressed: *
Operation: -
Pressed: 8
Pressed: #
3 - 8 = -5
```

Сценарий 3: Деление с дробной частью

1. Введите 8 / 3 (нажимая 8, * до появления знака деления, 3, #).
   Индикатор: 2.67 (десятичная точка горит после двойки).
   Консоль: Operation: / \n 8 / 3 = 2.67



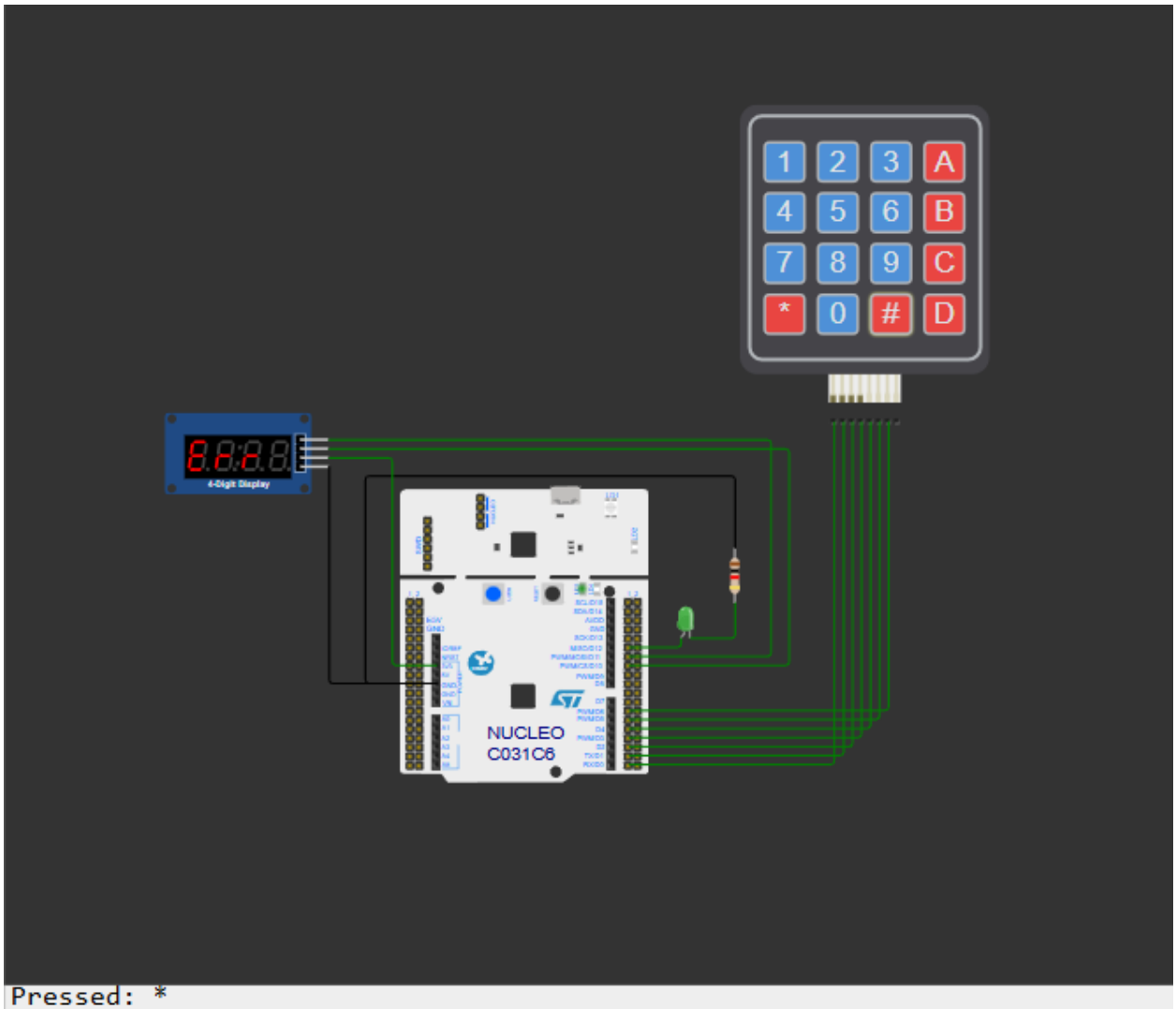

```
Calculator: digit, * (operation), digit, # (result)
Pressed: 8
Pressed: *
Operation: +
Pressed: *
Operation: -
Pressed: *
Operation: *
Pressed: *
Operation: /
Pressed: 3
Pressed: #
8 / 3 = 2.67
```

Сценарий 4: Обработка ошибки (деление на ноль)

1. Введите 5 / 0 и нажмите #.
   Индикатор: Err
   Консоль: 5 / 0 = Err (division by zero)

4.3. Сброс и новый расчет

Если на индикаторе отображается результат (например, 5), нажатие любой цифры (например, 1) автоматически очистит предыдущий расчет, и индикатор покажет 1 _ _ _, начиная новый ввод. Повторное нажатие # после получения результата также выполняет сброс.




```
Pressed: *
Operation: *
Pressed: *
Operation: /
Pressed: 0
Pressed: #
5 / 0 = Err (division by zero)
```

5. Ссылка на репозиторий с кодом

https://github.com/CodeCGremlin/ITMO/tree/main/embedded%20systems/l1